Causal Confidence Halting for Frozen Recursive Sequence Models

Anonymous ACL submission

Abstract

We study runtime control of frozen recursive sequence models, where a latent state is refined up to three times before each output token. Most tokens already carry the correct prediction after the base step. Adaptive Computation Time (Graves, 2016) and later halting methods train a gate inside the network, so their savings are predicted exit depths rather than executed calls. We present causal confidence halting, a two-stage threshold that stops a token on confidence using only signals available before the refinement it gates. Because high-confidence tokens are stable across depths, stopping there preserves the prediction a deeper step would have produced. On COGS with a frozen GRU encoder–decoder, the threshold retains the exact-match accuracy of the deepest fixed depth while reducing executed calls to 25.7% of that baseline on two seeds and 52.1% of the shallower e_1 baseline on the third. A preregistered confidence-bound criterion confirms non-inferiority on all three model seeds, and runtime interception verifies every counted call. At fixed depth, evolutionary step-size policies outperform the original update by up to $+2.1$pp at constant compute. We release the instrumented executor, per-evaluation search logs, and the full confirmation protocol.

1 Introduction

Recursive sequence models reapply a refinement operator to a latent state so that later iterations can repair earlier mistakes (Graves, 2016; Dehghani et al., 2019). On COGS semantic parsing (Kim and Linzen, 2020) with a frozen GRU encoder–decoder, the benefit is uneven because 87–91% of examples keep their prediction across the initial refinement while only 5–9% are repaired by it. Running the deepest configuration for every token therefore spends most of its compute on tokens that need no further work.

Adaptive Computation Time allocates variable thinking steps through a learned halting unit (Graves, 2016), PonderNet learns a probabilistic halting distribution (Banino et al., 2021), and depth-adaptive transformers exit per layer or per head (Elbayad et al., 2020; Schuster et al., 2022). These results assume the network can be retrained, and they express savings as a function of predicted exit depth. A frozen model admits neither assumption. No halting parameters can be fitted, and a predicted depth is not evidence that a call was skipped. The gap between predicted and executed compute is exactly where an adaptive method can report a saving it never delivered.

The observation that resolves this gap is local to each token: high-confidence predictions are the ones that stay fixed under refinement, so a confidence score computed before a step tells us whether that step can pay for itself. We present causal confidence halting, a two-stage threshold that reads only signals available before the refinement it gates. The earlier gate decides after the base step and the later gate after one refinement; each stops the token when the top prediction clears a preregistered threshold. The executor never computes a step it has decided to skip, so every saved refinement is a call the model truly avoids.

In contrast to learned halting (Graves, 2016; Banino et al., 2021) and confidence-based early exit (Schuster et al., 2022), our procedure changes what is measured as much as what is decided. Prior methods train a gate and infer cost from exit depths; ours freezes the entire model and obtains cost by intercepting the refinement module at runtime. The difference is checkable: forced-depth decoding reproduces frozen trajectories exactly, and a forced stop at the base-step gate executes zero refinement calls. Exact call counting therefore turns a compute ratio into an audited quantity.

On three frozen model seeds of COGS, the two-stage threshold retains the exact-match accuracy of each seed's best fixed depth while using 25.7% of the deepest baseline's calls on two seeds and

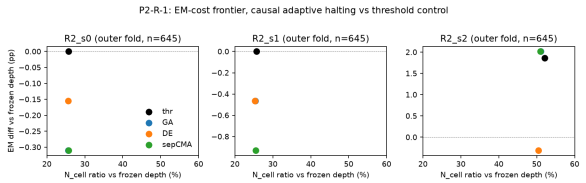


Figure 1: Causal confidence halting reaches the accuracy–compute frontier with a two-stage threshold. Per-seed held-out exact match against the ratio of executed recurrent-cell calls to each seed's frozen best fixed depth. Evolutionary halting policies cluster near the threshold: the confidence-bound rule accepts the threshold on all three seeds and only two of the nine searched policies. Curves connect policies evaluated at the same search budget, so horizontal differences measure actual skipped refinement calls rather than predicted exit depths.

52.1% of the shallower e_1 baseline on the third. A preregistered confidence-bound criterion, applied once to frozen policies on a held-out fold, confirms non-inferiority on all three seeds. The same accounting extends to the latent update itself: at fixed depth, evolutionary step-size policies outperform the original update by up to $+2.1$pp. Halting and step-size control are thus measured on a single scale of actually executed calls.

Our stance follows from where the gains come from. On frozen models, the controller-complexity budget is better spent on accounting and confirmation than on richer controllers, because the halting decision is already determined by a signal the base step emits. We therefore study two control dimensions, whether to continue and how to update, and keep them separate so that fixed-depth experiments isolate latent dynamics at constant compute before joint policies are tested on the accuracy–compute frontier of Figure 1. The mechanism, the equal-budget comparisons, and the confirmation protocol follow in the next two sections.

2 Related Work

Adaptive computation and early exit. Adaptive Computation Time allocates variable thinking steps to recurrent networks with a learned halting unit (Graves, 2016); PonderNet learns a probabilistic halting distribution (Banino et al., 2021); the Universal Transformer applies depth-adaptive self-attention (Dehghani et al., 2019), and depth-adaptive transformers exit per layer or per head (Elbayad et al., 2020). Confidence-based early exit with calibrated scores matches instance complexity to model depth (Schwartz et al., 2020), guarantees consistency between early-exit and full-model predictions (Schuster et al., 2021), and, closest to our token-level setting, allocates compute per generation timestep in language modeling with sequence-level risk control (Schuster et al., 2022). Each of these methods fits a gate or a halting distribution, whereas our model is frozen and no halting parameters are trained. All reported compute is the exact count of executed refinement calls, obtained by runtime interception rather than derived from exit depths. Anytime and meta-reasoning approaches trade deliberation against cost in a similar spirit (Zilberstein, 1996).

Recursive and iterative refinement. Repeated application of a transformation to a latent state underlies iterative deepening in sequence transduction, from learned halting in recurrent models (Graves, 2016) to repeated transformer passes (Dehghani et al., 2019). The empirical structure we exploit is that most examples are stable under additional iterations and a small fraction are repaired. We quantify it as four-way depth transitions (repair, harm, preserve, unresolved) and use the transition counts to design the stopping criterion.

Evolutionary computation for neural control. EC has been applied to neural weights (neuroevolution) (Stanley et al., 2019) and to controller optimization (Eiben and Smith, 2015). We restrict EC to a five-dimensional step-size policy on a frozen model, which separates search from representation learning. Our random-search control follows the recommendation that optimizer claims require equal-budget baselines (Eiben and Smith, 2015); our ablations follow mechanism-attribution logic common in analysis-driven research.

Confirmation protocols. Cluster-robust bootstrap is standard for grouped data (Cameron and Miller, 2015). We adopt a preregistered non-inferiority criterion: the lower 95% confidence bound of the accuracy difference must be ≥ -0.5pp. We evaluate frozen policies once on a held-out fold, following preregistration practice in empirical sciences. The confidence-bound rule and the confirmation stage matter because point-estimate classifications and inner-subset optima both overstate reliability in our own results.

3 Method

Runtime control in this paper has two parts: a halting gate decides which refinements to execute, and a step-size policy decides how strongly each executed refinement moves the latent state. Both read only causally available signals from a frozen GRU encoder–decoder, and both are measured by the same counter of executed cell calls. This section defines the base model, the gate, the counter, and the two policy families; §4 evaluates them against per-seed fixed-depth baselines.

3.1 Recursive GRU Seq2Seq Model

To make the control problem explicit, we first fix the model whose refinement steps are gated. The model is a GRU encoder–decoder (Cho et al., 2014; Sutskever et al., 2014) in which, at output position i, a base step produces a latent state $z_i^{(0)}$ (depth e_0). Up to $K{=}3$ refinement steps may follow, each applying a frozen GRU cell F to the current latent:

$$\begin{aligned}\Delta z_i^{(k)} &= F\big(\big[\,\mathrm{emb}(y_{i-1});\, c_i;\\ &\qquad e_{\mathrm{soft}}(z_i^{(k)})\,\big],\, z_i^{(k)}\big) - z_i^{(k)}, \qquad (1)\\ z_i^{(k+1)} &= z_i^{(k)} + \Delta z_i^{(k)},\end{aligned}$$

where c_i is the encoder context, $e_{\mathrm{soft}}(z)$ the expected embedding under the current output distribution, and $k \in \{0,\dots,K{-}1\}$ indexes refinement steps (depth e_1 after one step, e_3 after three). Six checkpoints (two model families $\times$ three seeds $s \in \{0,1,2\}$) are frozen; all experiments use family R2.

Empirical depth structure. On the gate-training split (n=2,419 per seed), the transition between depths $e_0 \to e_1$ decomposes into *preserve* (both correct; 87–91%), *repair* (wrong→correct; 5–9%), *unresolved* (both wrong; 3–5%), and *harm* (correct→wrong; 0.12–0.25%). The repair-to-harm ratio of roughly 20–70:1 means recursion is nearly harmless but benefits a minority of examples, which is exactly the structure under which conservative stopping is safe.

3.2 Causal Token-Level Adaptive Halting

Halting needs a decision that is available before the cost it avoids is incurred. At each token, after the base step (and again after the first refinement), a gate decides whether to stop. Crucially, the decision at each gate may only use signals computed *before* the refinements it gates; the executor never computes a step whose cost is not incurred. Gate 0 (after e_0) uses

$$\begin{aligned}\mathbf{x}_i^{(0)} = \big[&\mathrm{top}_1,\ \mathrm{margin},\ H,\ \overline{\mathrm{top}}_1,\\ &p_{75}(H_{\mathrm{hist}}),\ i,\ \mathrm{Var}(\mathrm{top}_1),\\ &\mathbb{1}[\mathrm{EOS}]\,\big] \in \mathbb{R}^8,\end{aligned} \qquad (2)$$

where H is the entropy of the current predictive distribution and the remaining entries summarize the decoded prefix; refinement proceeds iff $\sigma(\mathbf{w}_0^\top \mathbf{x}_i^{(0)} + b_0) \geq \tau_0$. Gate 1 (after e_1) uses an analogous 12-dimensional vector that additionally contains the same-state deltas $\Delta\mathrm{top}_1$, ΔH, $\Delta\mathrm{margin}$ and $\|z^{(1)} - z^{(0)}\|$, and gates the remaining two refinements. The threshold policy in our headline results is the special case "stop iff $\mathrm{top}_1 \geq \tau$" with $(\tau_0, \tau_1) = (0.85, 0.90)$, preregistered before any evaluation.

3.3 Actual Computation Accounting

A halting claim is only as strong as the counter behind it, so the compute metric counts calls rather than depths. The metric is

$$N_{\mathrm{cell}} = \#\{\text{base steps}\} + \#\{\text{refinement calls}\}, \qquad (3)$$

summed over a corpus and reported relative to the frozen per-seed best fixed depth (§4.1). Correctness is enforced by unit tests (forced stop/continue at each gate yields exactly 0/1/3 refinements per token) and by runtime interception that wraps the refinement module and asserts the counter equals the true call count; forced-depth decoding reproduces frozen training-time trajectories exactly (60/60 samples per depth).

3.4 Evolutionary Latent Step-Size Control

Separately from halting, we ask whether the update magnitude itself should be input-dependent. We replace (1) with

$$\begin{aligned}z_i^{(k+1)} &= z_i^{(k)} + \alpha_i^{(k)}\,\Delta z_i^{(k)},\\ \alpha_i^{(k)} &= \sigma\big(w_0 + w_1\tilde{H} + w_2\tilde{D}\\ &\qquad + w_3\tilde{A} + w_4\,k/K\big),\end{aligned} \qquad (4)$$

where $\tilde{\cdot}$ denotes standardization by statistics frozen from $\alpha{\approx}1$ rollouts of the first 64 examples of the inner training subset, H the current predictive entropy, $D = \|\Delta z\|_2/\sqrt{d}$ the previous displacement, and A the cosine between consecutive updates. Setting $\alpha \equiv 1$ recovers the original trajectories exactly (verified token-by-token, 128/128 per seed);

searches are warm-started from $w_0 = 8$, giving $\alpha \approx 0.9997$ rather than exactly 1. All features are causally available at decision time; no gold labels, oracle outcomes, or future-depth signals are used.

3.5 Joint Control

Combining both controllers raises a representation question: the two policies must be decoded by one procedure in which gate decisions precede the refinements they gate and step sizes modulate each executed refinement. A joint policy parameterizes both halting gates (24 dimensions, §3.2) and the step-size policy (5 dimensions, (4)), yielding a *nominal* 29-dimensional genotype. Two implementation details reduce the effective dimensionality, which we disclose here rather than overstate the genotype. First, the end-of-sequence indicator features of both gates are always zero at decision time under the current decoding logic (decoding terminates when EOS is predicted), so those two weights are inactive. Second, the joint path supplies a constant raw value for the direction-consistency feature A; after standardization this is a constant (not necessarily zero), so the corresponding weight is confounded with the step-size intercept rather than inactive. The nominal genotype thus has 29 coordinates, of which 27 are non-inactive, but only 26 are functionally independent degrees of freedom.

3.6 Statistical Criteria

Because a point estimate can clear a margin whose confidence bound falls below it, every claim in §4 is decided by an interval. All intervals are group-cluster paired bootstrap percentiles (Cameron and Miller, 2015) over structural groups of the task (1,000 resamples for development comparisons; 20,000 for held-out evaluation, fixed seeds). A policy is *non-inferior* to a baseline iff the **lower 95% confidence bound** of the exact-match (EM) difference is ≥ -0.5pp; point estimates alone do not establish non-inferiority. Search budgets are exactly 336 candidate evaluations per optimizer (including the initial population), with warm starts from the preregistered threshold policy and per-evaluation best-so-far logs.

4 Experiments

We evaluate the three claims of the mechanism in order. First, causal halting must retain accuracy while reducing executed calls. Second, step-size control must improve the latent update at constant compute. Third, joint policies must transfer from inner search to a held-out fold. Each comparison uses the same call counter and the same confidence-bound rule.

4.1 Setup

We use the COGS semantic parsing benchmark (Kim and Linzen, 2020) with a frozen gate split of the training distribution (2,419 examples per seed): inner folds (4/5 of groups) supply search fitness on group-balanced subsets (192–256 examples); a held-out fold (fold-4, 645 examples per seed, group-disjoint from the search folds) is used only for one-shot evaluation of frozen policies. Group keys, derived from the logical forms of the full training split, are used for fold construction, group-balanced sampling, and bootstrap resampling, but never as per-sample policy inputs. Per-seed frozen best fixed depths are e_3, e_3, e_1 for seeds 0–2; all EM differences and compute ratios are relative to each seed's own baseline. The sealed COGS test and generalization sets were never read in any experiment reported here. Optimizers (GA, DE, sep-CMA-ES, random search) share the budget of §3.6; the optimizer seed is fixed at 0, so optimizer comparisons are exploratory.

4.2 Causal Execution and Implementation Validation

A preregistered engineering audit confirmed: (i) forced-depth decoding matches frozen trajectories exactly (60/60 samples per depth, per seed); (ii) the compute counter equals the true base+refinement call count under runtime interception; (iii) a forced stop at gate 0 executes exactly zero refinement calls; (iv) encoder caching used for throughput leaves all decoded tokens, gate actions, and costs bit-identical. Throughput after caching is 24–42 examples/s on CPU.

4.3 Adaptive Halting Results (RQ1)

Causal confidence halting retains exact-match accuracy at a fraction of the executed compute (Table 1, Figure 1). The two-stage threshold is non-inferior on all three seeds while reducing the measured N_{cell} count of counted recurrent-cell invocations (§3.3) by 74–75% relative to the deepest baseline (seeds 0–1) and by 48% relative to the already shallow e_1 baseline while gaining 1.86pp EM (seed 2). The mechanism is consistent with §3.1: high-confidence tokens are stable across depths, so stopping on confidence is safe. Equal-budget evolutionary optimizers add at most 1–2% compute, and

Seed / baseline	Policy	ΔEM (pp)	95% CI (pp)	N_{cell}	Non-inf.
s_0 / e_3	threshold	+0.00	[0.00, 0.00]	0.2575	✓
	GA	−0.31	[−0.99, 0.00]	0.2563	×
	DE	−0.16	[−0.83, 0.47]	0.2563	×
	sep-CMA-ES	−0.31	[−0.99, 0.00]	0.2568	×
s_1 / e_3	threshold	+0.00	[0.00, 0.00]	0.2570	✓
	GA	−0.47	[−1.36, 0.00]	0.2537	×
	DE	−0.47	[−1.36, 0.00]	0.2530	×
	sep-CMA-ES	−0.93	[−2.33, −0.14]	0.2553	×
s_2 / e_1	threshold	+1.86	[−0.48, 6.82]	0.5212	✓
	GA	+2.02	[0.00, 6.95]	0.5103	✓
	DE	−0.31	[−1.00, 0.00]	0.5047	×
	sep-CMA-ES	+2.02	[0.00, 6.95]	0.5100	✓

Table 1: Confidence halting retains exact-match accuracy at a fraction of the executed compute. Held-out fold (n=645 per seed), relative to each seed's frozen best fixed depth (e_3 for seeds 0–1, e_1 for seed 2). Non-inferiority is decided by the preregistered rule $\text{LCB}_{95\%}(\Delta\text{EM}) \geq -0.5$pp, *not* by point estimates. The threshold policy is non-inferior on all three seeds; only 2 of 9 evolutionary policies pass, although 7 of 9 satisfy the point-estimate margin.

	Search (inner EM gain, pp)				Ablation (GA, s_0; §4.6)		
Seed	RS	GA	DE	sep-CMA	A: only k/K	B: only H	C: only D, A
s_0	+0.00	+0.52	+0.52	+0.00	+0.52	+0.00	+0.52
s_1	+1.04	+1.56	+1.04	+2.08	n/a	n/a	n/a
s_2	+0.52	+0.52	+1.04	+0.00	n/a	n/a	n/a

Table 2: Step-size policies improve fixed-depth exact match at constant compute. Fixed-depth (constant N_{cell}) improvements over the original $\alpha\approx1$ update on the inner subset (n=192); all searches use exactly 336 evaluations. A hand-designed linearly decaying schedule ($\alpha_k = 1.0 - 0.5k/K$, i.e., $[1.0, 0.833, 0.667]$ at depth 3) and fixed $\alpha \in \{0.75, 0.5, 0.25\}$ never exceed the $\alpha\approx1$ baseline. Ablations on s_0 restrict the policy input to a single feature group under the corrected masks of §4.6; the full-feature GA result corresponds to the GA column.

only two of the nine evolutionary policies satisfy the preregistered confidence-bound non-inferiority criterion, because most optimizer gains come with EM confidence intervals whose lower bounds fall below −0.5pp. GA and sep-CMA-ES converge to identical behavior on s_2 and are the two passing policies. On this model the halting problem is essentially solved by the threshold, and we did not expand the optimizer comparison further.

4.4 Fixed-Depth Evolutionary Latent Control (RQ2)

At fixed depth the compute is constant, so any EM change is a pure latent-dynamics effect. Evolutionary step-size policies outperform the original update on all seeds, by up to +2.08pp (Table 2, Figure 2). On every seed at least one optimizer beats same-budget random search, but no single optimizer does so consistently: on s_2, GA merely

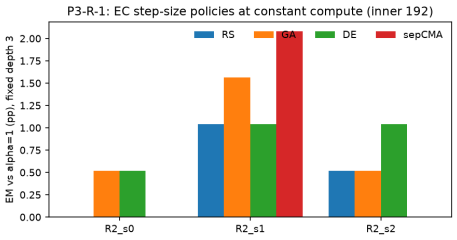


Figure 2: Step-size policies improve fixed-depth exact match at constant compute. Inner-subset EM gain (pp) over the original $\alpha\approx1$ update for random search and three evolutionary optimizers at equal budgets of 336 evaluations. Feature-group ablations on s_0 show that the time index alone and the dynamics pair alone each reproduce the full-feature gain, while entropy alone does not.

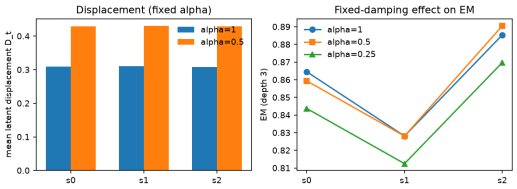


Figure 3: Refinement steps change the latent state less as depth grows. Per-step displacement and direction consistency of the latent trajectory under fixed damping on the frozen GRU decoder. The concentration of small displacements at later steps motivates a step-size policy that reads entropy, displacement, and direction consistency at decision time.

matches RS and sep-CMA falls below it. A hand-designed linearly decaying schedule over the three steps and all fixed step sizes fail to exceed the baseline. The corrected feature-group ablations on s_0 are decisive: a time-index-only policy (A) and a latent-dynamics-only policy (C, using D and A) each reproduce the full-feature GA gain exactly (+0.52pp), whereas an entropy-only policy (B) does not (+0.00pp). The full five-feature state is never necessary, and since dynamics features alone also suffice, the gain cannot be attributed to a pure time schedule either. Figure 3 shows the displacement and direction-consistency statistics that motivate these features.

4.5 Joint Control and One-Shot Evaluation (RQ3)

Joint search (27 active dimensions, §3) improves the inner compute ratio over the threshold by only 0.8–5.7%, so the decisive question is transfer to held-out data (Figure 4). The preregistered one-shot evaluation in Table 3 shows that the threshold

Seed	Threshold halting (J1) ΔEM (pp)	ratio	Joint EC policy (J4) ΔEM (pp)	95% CI (pp)	Non-inf.
s_0	$+0.00$	0.2575	$+0.00$	$[0.0, 0.0]$	✓
s_1	$+0.00$	0.2570	-0.31	$[-1.0, 0.0]$	×
s_2	$+1.86$	0.5212	-0.16	$[-0.6, 0.0]$	×

Table 3: One-shot confirmation separates retained gains from inner-subset gains. Held-out fold (n=645 per seed); CIs are 95% group-cluster bootstrap intervals of ΔEM (pp) vs. the frozen-depth baseline (20,000 resamples). J1 passes non-inferiority on all seeds, on s_2 only marginally, with 95% CI $[-0.50, +7.43]$pp, i.e., an LCB of -0.4975pp, only 0.0025pp above the bound. The best joint EC policy (selected by inner fitness) fails on s_1 and s_2, losing on s_2 the entire EM gain of the threshold policy (paired ΔEM -2.02pp vs. J1).

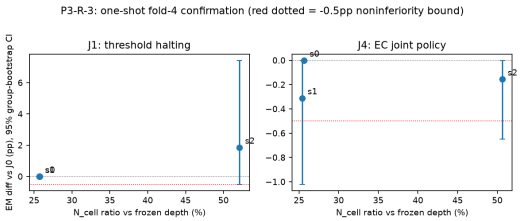


Figure 4: One-shot confirmation separates retained gains from inner-subset gains. Held-out ΔEM with 95% group-cluster bootstrap intervals for the threshold halting policy (J1) and the joint EC policy (J4) on each seed. The dotted line marks the preregistered -0.5pp non-inferiority bound; J1 stays above it on all three seeds, with the s_2 interval approaching the bound.

policy passes non-inferiority on all three seeds, on s_2 only marginally with an LCB of -0.4975pp, 0.0025pp above the bound. The jointly optimized policies, despite favorable inner results such as $+0.52$pp at equal cost on s_1, fail on s_1 and s_2, and on s_2 lose the entire $+1.86$pp held-out gain of the plain threshold. Inner-subset gains of low-dimensional EC controllers do not transfer to the accuracy–compute frontier.

4.6 Negative Results and Deviations

We report four findings as first-class results. First, the original 25%-compute claim in our internal history used a look-ahead executor and was invalidated; after rebuilding causally, the number happens to be similar but the evidence is entirely different. Second, EC latent control does not survive held-out evaluation on frozen models: on the ablated seed its inner gain is reproducible by either a time-index-only or a dynamics-only policy and cannot be attributed uniquely to a temporal schedule or to an input-dependent mechanism (§4.4). Third, the preregistered feature-group ablations B (only H) and C (only D, A) were initially executed with mismatched feature masks (B ran with only D; C ran with A and k/K), which an audit of the search code and logged masks revealed. The mislabeled runs are archived, and B and C were re-run with corrected masks at the preregistered budget; under the corrected masks, entropy alone does not reproduce the gain, while dynamics features alone do (Table 2). Fourth, a data-use audit shows that the held-out fold was previously used in an earlier internal development report of a different experiment line, and 51 sample IDs per seed overlapped with an earlier threshold-development subset. It was not used for P3-R genotype optimization, but "fully untouched" claims would be inaccurate (§*Limitations*). Additional deviations (a warm start outside preregistered bounds, an early-terminated CMA run later completed at full budget with unchanged results) are enumerated in the appendix with their resolutions.

5 Conclusion

On a frozen recursive GRU model, the reliable way to exploit uneven refinement benefit is a simple, causal, audited confidence threshold. It retains full accuracy at 25.7–25.8% of the deepest (e_3) compute on two seeds and 52.1% of the shallower e_1 baseline compute on the third, and it passes one-shot non-inferiority evaluation on all three seeds. Because the executor counts every base and refinement call, these ratios are measured savings rather than inferred depths. The same accounting shows where controller complexity pays: equal-budget evolutionary search adds at most 1–2% compute savings to halting, while fixed-depth step-size policies improve the latent update by up to $+2.1$pp. Our stance is that on frozen models the controller budget is better spent on accounting and confirmation than on richer controllers. Making latent refinement control transferable calls for changing the refinement operator or its training, rather than post-hoc search over a frozen one. We release the causal executor, exact call-count tests, equal-budget controls, mechanism ablations, and the one-shot confirmation protocol as minimum practice for adaptive-computation research.

Limitations

The held-out fold was used in a prior internal development report of a different experiment line and shares 51 sample IDs per seed with an ear-

lier threshold-development subset. It was not used for P3-R genotype optimization, so this evaluation is a one-shot test of frozen policies on a historically exposed split rather than confirmation on pristine untouched data. The sealed COGS test and generalization sets remain the only untouched data and were not used. Results come from one dataset, one small GRU architecture, depth ≤ 3, and single optimizer seeds; optimizer comparisons are exploratory. Ablations were run on one model seed as preregistered; they show the full input state is unnecessary in that seed, not that individual input features are never useful. Wall-clock latency does not shrink proportionally with call counts because gating overhead remains, and we therefore report call counts as the primary metric. N_{cell} counts decoder base and refinement cell invocations only; encoder computation, output projection, softmax, and the gates themselves are not equivalently counted, so it is not a total-FLOPs measure and latency cannot be inferred from it. Group keys used for fold construction, sampling, and bootstrap resampling were derived from the logical forms of the full training split; they never enter any policy as input, but fold structure is not fully independent of the label vocabulary.

References

Andrea Banino, Jan Balaguer, and Charles Blundell. 2021. Pondernet: Learning to ponder. In *ICML 2021 Workshop on Automated Machine Learning (AutoML)*.

A. Colin Cameron and Douglas L. Miller. 2015. A practitioner's guide to cluster-robust inference. *Journal of Human Resources*, 50(2):317–372.

Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of EMNLP*.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. 2019. Universal transformers. In *International Conference on Learning Representations (ICLR)*.

Agoston E. Eiben and James E. Smith. 2015. *Introduction to Evolutionary Computing*, 2nd edition. Springer.

Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. 2020. Depth-adaptive transformer. In *International Conference on Learning Representations (ICLR)*.

Alex Graves. 2016. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*.

Najoung Kim and Tal Linzen. 2020. COGS: A compositional generalization challenge based on semantic interpretation. In *Proceedings of EMNLP*, pages 9087–9105.

Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. 2022. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 17456–17472.

Tal Schuster, Adam Fisch, Tommi Jaakkola, and Regina Barzilay. 2021. Consistent accelerated inference via confident adaptive transformers. In *Proceedings of EMNLP*, pages 4962–4979.

Roy Schwartz, Gabriel Stanovsky, Swabha Swayamdipta, Jesse Dodge, and Noah A. Smith. 2020. The right tool for the job: Matching model and instance complexities. In *Proceedings of ACL*, pages 6640–6651.

Kenneth O. Stanley, Jeff Clune, Joel Lehman, and Risto Miikkulainen. 2019. Designing neural networks through neuroevolution. *Nature Machine Intelligence*, 1:24–35.

Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Shlomo Zilberstein. 1996. Using anytime algorithms in intelligent systems. *AI Magazine*, 17(3):73–83.

A Reproducibility Materials

All experiments are preregistered in the repository before execution. Each search logs all 336 evaluations with best-so-far curves, genotypes, and applied feature masks. A 79-check automated audit validates selected protocol-level and result-summary invariants against raw experiment artifacts, and a separate manuscript-specific audit of 132 checks cross-verifies, row by row, the numbers printed in this paper (including the confidence-bound non-inferiority classifications and the ablation masks) against the authoritative JSON results. Corrected ablation reruns were launched from a modified working tree based on commit 87842c9. The run manifest records the HEAD commit rather than a committed snapshot of the modified script, so we additionally release a provenance file with the SHA-256 of the exact script executed, and the applied masks are recorded in each result JSON.

Asset SHA-256 hashes, exact commands, environment versions, and the full deviation register (including the archived mislabeled ablation runs) are included in the released materials.

B Full Confidence Intervals for Development Comparisons

Table 1 reports the complete 95% CIs for the held-out fold. For the inner-search comparisons of Table 2 (fitness on the group-balanced inner subset), per-evaluation histories and best-so-far curves are released as JSON; the confirmation per-sample logs (JSONL) and the frozen summary file from which Table 3 is generated are included verbatim in the results archive.